

Солдатов Владимир ИСИП-406 Практическая работа 3

1. Класс Phone.

Создайте класс Phone, который содержит переменные number, model и weight.

Создайте три экземпляра этого класса.

Выведите на консоль значения их переменных.

Добавить в класс Phone методы: receiveCall, имеет один параметр – имя звонящего. Выводит на консоль сообщение “Звонит {name}”. Метод getNumber – возвращает номер телефона. Вызвать эти методы для каждого из объектов.

Добавить конструктор в класс Phone, который принимает на вход три параметра для инициализации переменных класса - number, model и weight.

Добавить конструктор, который принимает на вход два параметра для инициализации переменных класса - number, model.

Добавить конструктор без параметров.

Вызвать из конструктора с тремя параметрами конструктор с двумя.

Создать метод sendMessage с аргументами переменной длины. Данный метод принимает на вход номера телефонов, которым будет отправлено сообщение. Метод выводит на консоль номера этих телефонов.

C: > Users > SoW > Desktop > Vova > Практическая_работа_3 > flutter_3_1.da

Run | Debug

```
1 void main() {  
2     Phone p1 = Phone("12345", "iPhone", 200);  
3     Phone p2 = Phone.withTwo("54321", "Samsung");  
4     Phone p3 = Phone.empty();  
5  
6     print(p1.number);  
7     print(p1.model);  
8     print(p1.weight);  
9  
10    print(p2.number);  
11    print(p2.model);  
12    print(p2.weight);  
13  
14    print(p3.number);  
15    print(p3.model);  
16    print(p3.weight);  
17  
18    p1.receiveCall("Alice");  
19    p2.receiveCall("Bob");  
20    p3.receiveCall("Charlie");  
21  
22    print(p1.getNumber());  
23    print(p2.getNumber());  
24    print(p3.getNumber());  
25  
26    p1.sendMessage(["11111", "22222", "33333"]);  
27    p2.sendMessage(["44444", "55555"]);  
28 }  
29
```

C:\Users\Sow\Desktop> vova > Практическая_работа_3 > flutter_3_1.dart >

```
29
30 class Phone {
31     String number;
32     String model;
33     double weight;
34
35     // Конструктор с 3 параметрами
36     Phone(this.number, this.model, this.weight);
37
38     // Конструктор с 2 параметрами
39     Phone.withTwo(this.number, this.model) : weight = 0;
40
41     // Пустой конструктор
42     Phone.empty()
43         : number = "",
44         : model = "",
45         : weight = 0;
46
47     void receiveCall(String name) {
48         print("Звонит $name");
49     }
50
51     String getNumber() {
52         return number;
53     }
54
55     void sendMessage(List<String> numbers) {
56         for (var n in numbers) {
57             print("Сообщение отправлено на $n");
58         }
59     }

```

37
38

// Конструктор с 2 параметрами

39

Phone.withType<this>(this.number, this.model); // weight = 0.0

ПРОБЛЕМЫ

ВЫХОДНЫЕ ДАННЫЕ

КОНСОЛЬ ОТЛАДКИ

ТЕРМИНАЛ

ПОРТЫ

PS C:\> dart run flutter_3_1.dart

12345

iPhone

200.0

54321

Samsung

0.0

0.0

Звонит Alice

Звонит Bob

Звонит Charlie

12345

54321

Сообщение отправлено на 11111

Сообщение отправлено на 22222

Сообщение отправлено на 33333

Сообщение отправлено на 44444

Сообщение отправлено на 55555

PS C:\> █

2. Класс Матрица

Создать класс "Матрица". Класс должен иметь следующие переменные:

двумерный массив вещественных чисел;

количество строк и столбцов в матрице.

Класс должен иметь следующие методы:

сложение с другой матрицей;

умножение на число;

вывод на печать;

умножение матриц.

```
C: > Users > SoW > Desktop > Vova > Практическая_работа_3 > flutter_3_2.dart > main
Run | Debug
1 void main() {
2   Matrix a = Matrix([
3     [1, 2],
4     [3, 4]
5   ]);
6
7   Matrix b = Matrix([
8     [5, 6],
9     [7, 8]
10  ]);
11
12  print("Matrix A:");
13  a.printMatrix();
14
15  print("Matrix B:");
16  b.printMatrix();
17
18  print("A + B:");
19  a.add(b).printMatrix();
20
21  print("A * 2:");
22  a.multiplyByNumber(2).printMatrix();
23
24  print("A * B:");
25  a.multiplyMatrix(b).printMatrix();
26 }
27
```

```
C: > Users > SoW > Desktop > Vova > Практическая_работа_3 > flutter_3_2.dart >
1  void main() {
25  |   a.multiplyMatrix(b).printMatrix();
26  }
27
28  class Matrix {
29  |   List<List<double>> data;
30  |   int rows;
31  |   int cols;
32
33  |   Matrix(this.data)
34  |   |   : rows = data.length,
35  |   |   |   cols = data.isNotEmpty ? data[0].length : 0;
36
37  |   /// Сложение матриц
38  |   Matrix add(Matrix other) {
39  |   |   List<List<double>> result = List.generate(rows, (i) {
40  |   |   |   return List.generate(cols, (j) {
41  |   |   |   |   return data[i][j] + other.data[i][j];
42  |   |   |   |   }); // List.generate
43  |   |   |   }); // List.generate
44
45  |   |   return Matrix(result);
46  |   }
47
48  |   /// Умножение на число
49  |   Matrix multiplyByNumber(double number) {
50  |   |   List<List<double>> result = List.generate(rows, (i) {
51  |   |   |   return List.generate(cols, (j) {
52  |   |   |   |   return data[i][j] * number;
53  |   |   |   |   }); // List.generate
```

C: > Users > SoW > Desktop > Vova > Практическая_работа_3 > flutter_3_2.dart > main

```
28 class Matrix {  
48     /// Умножение на число  
49     Matrix multiplyByNumber(double number) {  
50         List<List<double>> result = List.generate(rows, (i) {  
51             return List.generate(cols, (j) {  
52                 return data[i][j] * number;  
53             }); // List.generate  
54         }); // List.generate  
55  
56         return Matrix(result);  
57     }  
58  
59     /// Умножение матриц  
60     Matrix multiplyMatrix(Matrix other) {  
61         List<List<double>> result =  
62             List.generate(rows, (i) => List.filled(other.cols, 0));  
63  
64         for (int i = 0; i < rows; i++) {  
65             for (int j = 0; j < other.cols; j++) {  
66                 for (int k = 0; k < cols; k++) {  
67                     result[i][j] += data[i][k] * other.data[k][j];  
68                 }  
69             }  
70         }  
71  
72         return Matrix(result);  
73     }  
74 }
```

```
C: > Users > SoW > Desktop > Vova > Практическая_работа_3 > flutter_3_2.dart > main
28   class Matrix {
60     Matrix multiplyMatrix(Matrix other) {
68         // ...
69     }
70 }
71
72     return Matrix(result);
73 }
74
75     /// Печать матрицы
76     void printMatrix() {
77         for (var row in data) {
78             print(row);
79         }
80         print("");
81     }
82 }
```

```
ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ    ТЕРМИНАЛ    ПОРТЫ

PS C:\> dart run flutter_3_2.dart
Matrix A:
[1.0, 2.0]
[3.0, 4.0]

Matrix B:
[5.0, 6.0]
[7.0, 8.0]

A + B:
[6.0, 8.0]
[10.0, 12.0]

A * 2:
[2.0, 4.0]
[2.0, 4.0]
[2.0, 4.0]
[6.0, 8.0]

A * B:
[19.0, 22.0]
[43.0, 50.0]

PS C:\> 
```


3. Рекурсивный вывод чисел

Даны два целых числа A и B. Выведите все числа от A до B включительно, в порядке возрастания, если $A < B$, или в порядке убывания в противном случае. Использовать рекурсию.

```
C: > Users > SoW > Desktop > Vova > Практическая_ работа_3 > flutter_3_3.dart > printRange
Run | Debug
1 void main() {
2   printRange(3, 8);
3   printRange(10, 5);
4 }
5
6 void printRange(int a, int b) {
7   if (a == b) {
8     print(a);
9     return;
10  }
11
12   if (a < b) {
13     print(a);
14     printRange(a + 1, b); // <- вот здесь рекурсивный вызов
15   } else {
16     print(a);
17     printRange(a - 1, b); // <- вот здесь рекурсивный вызов
18   }
19 }

ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ    ТЕРМИНАЛ    ПОРТЫ
PS C:\> dart run flutter_3_3.dart
3
4
5
6
7
8
10
9
8
7
6
5
PS C:\> 
```

4. Наследование Student, Aspirant

Создайте пример наследования, реализуйте класс Student и класс Aspirant, аспирант отличается от студента наличием некой научной работы.

Класс Student содержит переменные: String firstName, lastName, group. А также, double averageMark, содержащую среднюю оценку.

Создать метод getScholarship() для класса Student, который возвращает сумму стипендии. Если средняя оценка студента равна 5, то сумма 2000 руб, иначе 1900.

Переопределить этот метод в классе Aspirant. Если средняя оценка аспиранта равна 5, то сумма 2500 руб, иначе 2200.

Создать массив типа Student, содержащий объекты класса Student и Aspirant.

Вызвать метод getScholarship() для каждого элемента массива.

```
C: > Users > SoW > Desktop > Vova > Практическая_ работа_3 > flutter_3_4.dart > main
Run | Debug
1 void main() {
2     List<Student> students = [
3         Student("Ivan", "Petrov", "A1", 5),
4         Student("Anna", "Sidorova", "B2", 4.5),
5         Aspirant("Pavel", "Ivanov", "C3", 5),
6         Aspirant("Elena", "Smirnova", "D4", 4.8),
7     ];
8
9     for (var s in students) {
10         print("${s.firstName} ${s.lastName}: ${s.getScholarship()}");
11     }
12 }
13
14 class Student {
15     String firstName;
16     String lastName;
17     String group;
18     double averageMark;
19
20     Student(this.firstName, this.lastName, this.group, this.averageMark);
21
22     int getScholarship() {
23         if (averageMark == 5) {
24             return 2000;
25         }
26         return 1900;
27     }
28 }
29
30 class Aspirant extends Student {
31     String scientificWork;
```

```
flutter_3_4.dart X
C: > Users > SoW > Desktop > Vova > Практическая_ работа_3 > flutter_3_4.dart > main
14  class Student {
22      int getScholarship() {
24          return 2000;
25      }
26      return 1900;
27  }
28  }
29
30  class Aspirant extends Student {
31      String scientificWork;
32
33      Aspirant(String firstName, String lastName, String group, double averageMark,
34          {this.scientificWork = ""})
35          : super(firstName, lastName, group, averageMark);
36
37      @override
38      int getScholarship() {
39          if (averageMark == 5) {
40              return 2500;
41          }
42          return 2200;
43      }
44  }
```

```
41      }
42      return 2200;
43  }
44  }
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```
PS C:\> dart run flutter_3_4.dart
Ivan Petrov: 2000
Anna Sidorova: 1900
Pavel Ivanov: 2500
Elena Smirnova: 2200
PS C:\> 
```